

TivaC-Datasheet

Table 1: Performance

Feature	Description
Core	ARM Cortex-M4F processor core
Performance	80-MHz operation; 100 DMIPS performance
Flash	256 KB single-cycle Flash memory
System SRAM	32 KB single-cycle SRAM
EEPROM	2KB of EEPROM
Internal ROM	Internal ROM loaded with TivaWare™ for C Series software

Table 2: Communication Interfaces

Feature	Description
Universal Asynchronous Receivers/Transmitter (UART)	Eight UARTs
Synchronous Serial Interface (SSI)	Four SSI modules
Inter-Integrated Circuit (I²C)	Four I²C modules with four transmission speeds including high-speed mode
Controller Area Network (CAN)	Two CAN 2.0 A/B controllers
Universal Serial Bus (USB)	USB 2.0 OTG/Host/Device

Table 3: System Integration

Feature	Description
Micro Direct Memory Access (μDMA)	ARM® PrimeCell® 32-channel configurable μDMA controller
General-Purpose Timer (GPTM)	Six 16/32-bit GPTM blocks and six 32/64-bit Wide GPTM blocks
Watchdog Timer (WDT)	Two watchdog timers
Hibernation Module (HIB)	Low-power battery-backed Hibernation module
General-Purpose Input/Output (GPIO)	Six physical GPIO blocks

Table 4: Advanced Motion Control

Feature	Description
Pulse Width Modulator (PWM)	Two PWM modules, each with four PWM generator blocks and a control block, for a total of 16 PWM outputs.
Quadrature Encoder Interface (QEI)	Two QEI modules

Table 5: Analog Support

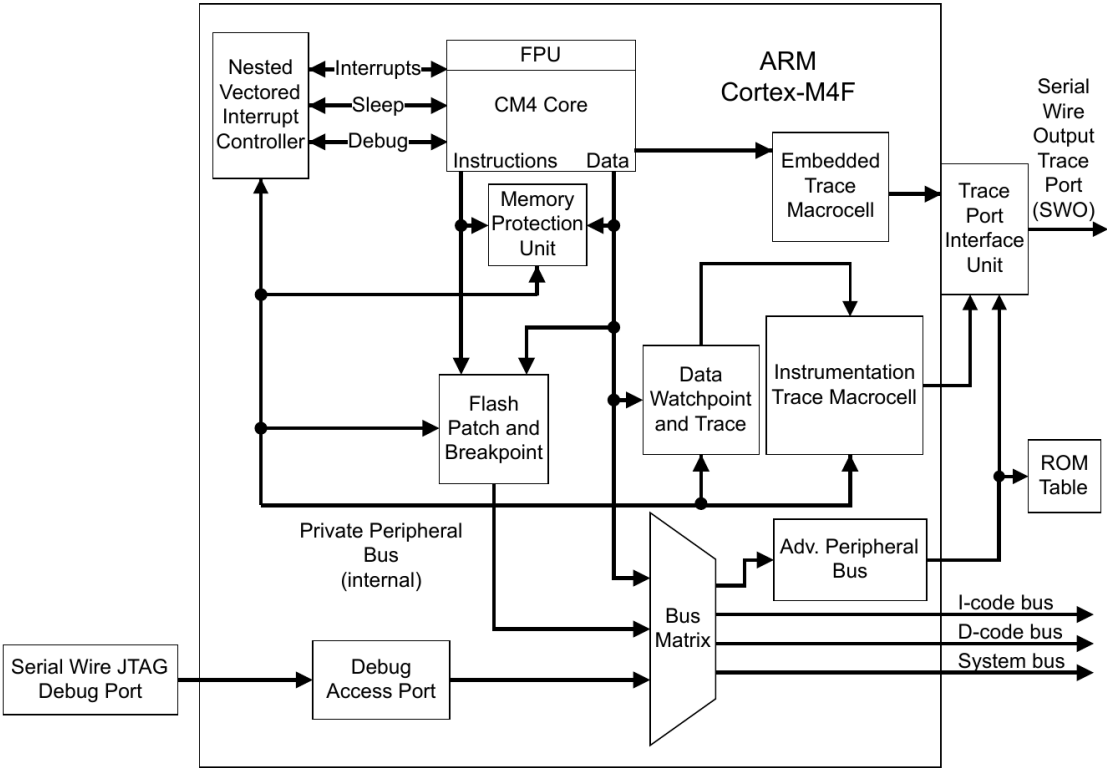
Feature	Description
Analog-to-Digital Converter (ADC)	Two 12-bit ADC modules, each with a maximum sample rate of one million samples/second
Analog Comparator Controller	Two independent integrated analog comparators
Digital Comparator	16 digital comparators
JTAG and Serial Wire Debug (SWD)	One JTAG module with integrated ARM SWD

Table 6: Package Information

Feature	Description
Package	64-pin LQFP
Operating Range (Ambient)	Industrial (-40°C to 85°C) temperature range Extended (-40°C to 105°C) temperature range

CPU Block Diagram

Figure 2-1. CPU Block Diagram



Processor Mode and Privilege Levels for Software Execution

The Cortex-M4F has two modes of operation:

- Thread mode
Used to execute application software. The processor enters Thread mode when it comes out of reset.
- Handler mode
Used to handle exceptions. When the processor has finished exception processing, it returns to Thread mode.

In addition, the Cortex-M4F has two privilege levels:

- Unprivileged
In this mode, software has the following restrictions:
 - Limited access to the MSR and MRS instructions and no use of the CPS instruction
 - No access to the system timer, NVIC, or system control block
 - Possibly restricted access to memory or peripherals
- Privileged
 - In this mode, software can use all the instructions and has access to all resources.
 - In Thread mode, the CONTROL register (see page 88) controls whether software execution is privileged or unprivileged. In Handler mode, software execution is always privileged.

Only privileged software can write to the CONTROL register to change the privilege level for software execution in Thread mode. Unprivileged software can use the SVC instruction to make a supervisor call to transfer control to privileged software.

Memory Map

Table 1: Memory

Start	End	Description	For details, see page ...
0x0000.0000	0x0003.FFFF	On-chip Flash	540
0x0004.0000	0x1FFF.FFFF	Reserved	-
0x2000.0000	0x2000.7FFF	Bit-banded on-chip SRAM	525
0x2000.8000	0x21FF.FFFF	Reserved	-
0x2200.0000	0x220F.FFFF	Bit-band alias of bit-banded on-chip SRAM starting at 0x2000.0000	525
0x2210.0000	0x3FFF.FFFF	Reserved	-

Table 2: Peripherals

Start	End	Description	For details, see page ...
0x4000.0000	0x4000.0FFF	Watchdog timer 0	776
0x4000.1000	0x4000.1FFF	Watchdog timer 1	776
0x4000.2000	0x4000.3FFF	Reserved	-
0x4000.4000	0x4000.4FFF	GPIO Port A	658

Start	End	Description	For details, see page ...
0x4000.5000	0x4000.5FFF	GPIO Port B	658
0x4000.6000	0x4000.6FFF	GPIO Port C	658
0x4000.7000	0x4000.7FFF	GPIO Port D	658
0x4000.8000	0x4000.8FFF	SSI0	967
0x4000.9000	0x4000.9FFF	SSI1	967
0x4000.A000	0x4000.AFFF	SSI2	967
0x4000.B000	0x4000.BFFF	SSI3	967
0x4000.C000	0x4000.CFFF	UART0	903
0x4000.D000	0x4000.DFFF	UART1	903
0x4000.E000	0x4000.EFFF	UART2	903
0x4000.F000	0x4000.FFFF	UART3	903
0x4001.0000	0x4001.0FFF	UART4	903
0x4001.1000	0x4001.1FFF	UART5	903
0x4001.2000	0x4001.2FFF	UART6	903
0x4001.3000	0x4001.3FFF	UART7	903
0x4001.4000	0x4001.FFFF	Reserved	-

Start	End	Description	For details, see page ...
0x4002.0000	0x4002.0FFF	I ² C 0	1017
0x4002.1000	0x4002.1FFF	I ² C 1	1017
0x4002.2000	0x4002.2FFF	I ² C 2	1017
0x4002.3000	0x4002.3FFF	I ² C 3	1017
0x4002.4000	0x4002.4FFF	GPIO Port E	658
0x4002.5000	0x4002.5FFF	GPIO Port F	658
0x4002.6000	0x4002.7FFF	Reserved	-
0x4002.8000	0x4002.8FFF	PWM 0	1240
0x4002.9000	0x4002.9FFF	PWM 1	1240
0x4002.A000	0x4002.BFFF	Reserved	-
0x4002.C000	0x4002.CFFF	QEIO	1310
0x4002.D000	0x4002.DFFF	QEI1	1310
0x4002.E000	0x4002.FFFF	Reserved	-
0x4003.0000	0x4003.0FFF	16/32-bit Timer 0	725
0x4003.1000	0x4003.1FFF	16/32-bit Timer 1	725
0x4003.2000	0x4003.2FFF	16/32-bit Timer 2	725
0x4003.3000	0x4003.3FFF	16/32-bit Timer 3	725
0x4003.4000	0x4003.4FFF	16/32-bit Timer 4	725
0x4003.5000	0x4003.5FFF	16/32-bit Timer 5	725
0x4003.6000	0x4003.6FFF	32/64-bit Timer 0	725
0x4003.7000	0x4003.7FFF	32/64-bit Timer 1	725
0x4003.8000	0x4003.8FFF	ADC0	818
0x4003.9000	0x4003.9FFF	ADC1	818
0x4003.A000	0x4003.BFFF	Reserved	-
0x4003.C000	0x4003.CFFF	Analog Comparators	1220
0x4003.D000	0x4003.FFFF	Reserved	-
0x4004.0000	0x4004.0FFF	CAN0 Controller	1067
0x4004.1000	0x4004.1FFF	CAN1 Controller	1067
0x4004.2000	0x4004.BFFF	Reserved	-
0x4004.C000	0x4004.CFFF	32/64-bit Timer 2	725
0x4004.D000	0x4004.DFFF	32/64-bit Timer 3	725
0x4004.E000	0x4004.EFFF	32/64-bit Timer 4	725
0x4004.F000	0x4004.FFFF	32/64-bit Timer 5	725
0x4005.0000	0x4005.0FFF	USB	1114
0x4005.1000	0x4005.7FFF	Reserved	-
0x4005.8000	0x4005.8FFF	GPIO Port A (AHB aperture)	658
0x4005.9000	0x4005.9FFF	GPIO Port B (AHB aperture)	658
0x4005.A000	0x4005.AFFF	GPIO Port C (AHB aperture)	658
0x4005.B000	0x4005.BFFF	GPIO Port D (AHB aperture)	658
0x4005.C000	0x4005.CFFF	GPIO Port E (AHB aperture)	658
0x4005.D000	0x4005.DFFF	GPIO Port F (AHB aperture)	658
0x4005.E000	0x400A.EFFF	Reserved	-
0x400A.F000	0x400A.FFFF	EEPROM and Key Locker	540
0x400B.0000	0x400F.8FFF	Reserved	-
0x400F.9000	0x400F.9FFF	System Exception Module	485
0x400F.A000	0x400F.BFFF	Reserved	-
0x400F.C000	0x400F.CFFF	Hibernation Module	505
0x400F.D000	0x400F.DFFF	Flash memory control	540
0x400F.E000	0x400F.EFFF	System control	231

Start	End	Description	For details, see page ...
0x400F.F000	0x400F.FFFF	μDMA	606
0x4010.0000	0x41FF.FFFF	Reserved	-
0x4200.0000	0x43FF.FFFF	Bit-banded alias of 0x4000.0000 through 0x400F.FFFF	-
0x4400.0000	0xDFFF.FFFF	Reserved	-

Table 3: Private Peripheral Bus

Start	End	Description	For details, see page ...
0xE000.0000	0xE000.0FFF	Instrumentation Trace Macrocell (ITM)	71
0xE000.1000	0xE000.1FFF	Data Watchpoint and Trace (DWT)	71
0xE000.2000	0xE000.2FFF	Flash Patch and Breakpoint (FPB)	71
0xE000.3000	0xE000.DFFF	Reserved	-
0xE000.E000	0xE000.EFFF	Cortex-M4F Peripherals (SysTick, NVIC, MPU, FPU and SCB)	134
0xE000.F000	0xE003.FFFF	Reserved	-
0xE004.0000	0xE004.0FFF	Trace Port Interface Unit (TPIU)	72
0xE004.1000	0xE004.1FFF	Embedded Trace Macrocell (ETM)	71
0xE004.2000	0xFFFF.FFFF	Reserved	-

Instruction Set Summary

Mnemonic	Operands	Brief Description	Flags
ADC, ADCS	{Rd,} Rn, Op2	Add with carry	N, Z, C, V
ADD, ADDS	{Rd,} Rn, Op2	Add	N, Z, C, V
ADD, ADDW	{Rd,} Rn , #imm12	Add	-
ADR	Rd, label	Load PC-relative address	-
AND, ANDS	{Rd,} Rn, Op2	Logical AND	N, Z, C
ASR, ASRS	Rd, Rm, <Rs \ #n>	Arithmetic shift right	N, Z, C
B	label	Branch	-
BFC	Rd, #lsb, #width	Bit field clear	-
BFI	Rd, Rn, #lsb, #width	Bit field insert	-
BIC, BICS	{Rd,} Rn, Op2	Bit clear	N, Z, C
BKPT	#imm	Breakpoint	-
BL	label	Branch with link	-
BLX	Rm	Branch indirect with link	-
BX	Rm	Branch indirect	-
CBNZ	Rn, label	Compare and branch if non-zero	-
CBZ	Rn, label	Compare and branch if zero	-
CLREX	-	Clear exclusive	-
CLZ	Rd, Rm	Count leading zeros	-
CMN	Rn, Op2	Compare negative	N, Z, C, V
CMP	Rn, Op2	Compare	N, Z, C, V
CPSID	i	Change processor state, disable interrupts	-
CPSIE	i	Change processor state, enable interrupts	-
DMB	-	Data memory barrier	-
DSB	-	Data synchronization barrier	-
EOR, EORS	{Rd,} Rn, Op2	Exclusive OR	N, Z, C
ISB	-	Instruction synchronization barrier	-
IT	-	If-Then condition block	-
LDM	Rn{!}, reglist	Load multiple registers, increment after	-
LDMDB, LDMEA	Rn{!}, reglist	Load multiple registers, decrement before	-
LDMFD, LDMIA	Rn{!}, reglist	Load multiple registers, increment after	-
LDR	Rt, [Rn, #offset]	Load register with word	-
LDRB, LDRBT	Rt, [Rn, #offset]	Load register with byte	-
LDRD	Rt, Rt2, [Rn, #offset]	Load register with two bytes	-
LDREX	Rt, [Rn, #offset]	Load register exclusive	-
LDREXB	Rt, [Rn]	Load register exclusive with byte	-
LDREXH	Rt, [Rn]	Load register exclusive with halfword	-
LDRH, LDRHT	Rt, [Rn, #offset]	Load register with halfword	-
LDRSB, LDRSBT	Rt, [Rn, #offset]	Load register with signed byte	-
LDRSH, LDRSHT	Rt, [Rn, #offset]	Load register with signed halfword	-
LDRT	Rt, [Rn, #offset]	Load register with word	-
LSL, LSLs	Rd, Rm, <Rs \ #n>	Logical shift left	N, Z, C
LSR, LSRS	Rd, Rm, <Rs \ #n>	Logical shift right	N, Z, C
MLA	Rd, Rn, Rm, Ra	Multiply with accumulate, 32-bit result	-

Mnemonic	Operands	Brief Description	Flags
MLS	Rd, Rn, Rm, Ra	Multiply and subtract, 32-bit result	-
MOV, MOVS	Rd, Op2	Move	N, Z, C
MOV, MOVW	Rd, #imm16	Move 16-bit constant	N, Z, C
MOVT	Rd, #imm16	Move top	-
MRS	Rd, spec_reg	Move from special register to general register	-
MSR	spec_reg, Rm	Move from general register to special register	N, Z, C, V
MUL, MULS	{Rd,} Rn, Rm	Multiply, 32-bit result	N, Z
MVN, MVNS	Rd, Op2	Move NOT	N, Z, C
NOP	-	No operation	-
ORN, ORNS	{Rd,} Rn, Op2	Logical OR NOT	N, Z, C
ORR, ORRS	{Rd,} Rn, Op2	Logical OR	N, Z, C
PKHTB, PKHBT	{Rd,} Rn, Rm, Op2	Pack halfword	-
POP	reglist	Pop registers from stack	-
PUSH	reglist	Push registers onto stack	-
QADD	{Rd,} Rn, Rm	Saturating add	Q
QADD16	{Rd,} Rn, Rm	Saturating add 16	-
QADD8	{Rd,} Rn, Rm	Saturating add 8	-
QASX	{Rd,} Rn, Rm	Saturating add and subtract with exchange	-
QDADD	{Rd,} Rn, Rm	Saturating double and add	Q
QDSUB	{Rd,} Rn, Rm	Saturating double and subtract	Q
QSAX	{Rd,} Rn, Rm	Saturating subtract and add with exchange	-
QSUB	{Rd,} Rn, Rm	Saturating subtract	Q
QSUB16	{Rd,} Rn, Rm	Saturating subtract 16	-
QSUB8	{Rd,} Rn, Rm	Saturating subtract 8	-
RBIT	Rd, Rn	Reverse bits	-
REV	Rd, Rn	Reverse byte order in a word	-
REV16	Rd, Rn	Reverse byte order in each halfword	-
REVSH	Rd, Rn	Reverse byte order in bottom halfword and sign extend	-
ROR, RORS	Rd, Rm, <Rs \ #n>	Rotate right	N, Z, C
RRX, RRXS	Rd, Rm	Rotate right with extend	N, Z, C
RSB, RSBS	{Rd,} Rn, Op2	Reverse subtract	N, Z, C, V
SADD16	{Rd,} Rn, Rm	Signed add 16	GE
SADD8	{Rd,} Rn, Rm	Signed add 8	GE
SASX	{Rd,} Rn, Rm	Signed add and subtract with exchange	GE
SBC, SBCS	{Rd,} Rn, Op2	Subtract with carry	N, Z, C, V
SBFX	Rd, Rn, #lsb, #width	Signed bit field extract	-
SDIV	{Rd,} Rn, Rm	Signed divide	-
SEL	{Rd,} Rn, Rm	Select bytes	-
SEV	-	Send event	-
SHADD16	{Rd,} Rn, Rm	Signed halving add 16	-
SHADD8	{Rd,} Rn, Rm	Signed halving add 8	-
SHASX	{Rd,} Rn, Rm	Signed halving add and subtract with exchange	-
SHSAX	{Rd,} Rn, Rm	Signed halving subtract and add with exchange	-
SHSUB16	{Rd,} Rn, Rm	Signed halving subtract 16	-
SHSUB8	{Rd,} Rn, Rm	Signed halving subtract 8	-
SMLABB, SMLABT, SMLATB, SMLATT	Rd, Rn, Rm, Ra	Signed multiply accumulate long (halfwords)	Q
SMLAD, SMLADX	Rd, Rn, Rm, Ra	Signed multiply accumulate dual	Q
SMLAL	RdLo, RdHi, Rn, Rm	Signed multiply with accumulate (32x32+64), 64-bit result	-
SMLALBB, SMLALBT, SMLALTB, SMLALTT	RdLo, RdHi, Rn, Rm	Signed multiply accumulate long (halfwords)	-
SMLALD, SMLALDX	RdLo, RdHi, Rn, Rm	Signed multiply accumulate long dual	-
SMLAWB, SMLAWT	Rd, Rn, Rm, Ra	Signed multiply accumulate, word by halfword	Q
SMLSD, SMLSDX	Rd, Rn, Rm, Ra	Signed multiply subtract dual	Q
SMLSLD, SMLSLDX	RdLo, RdHi, Rn, Rm	Signed multiply subtract long dual	-
SMMLA	Rd, Rn, Rm, Ra	Signed most significant word multiply accumulate	-
SMMLS, SMMLR	Rd, Rn, Rm, Ra	Signed most significant word multiply subtract	-
SMMUL, SMMULR	{Rd,} Rn, Rm	Signed most significant word multiply	-
SMUAD, SMUADX	{Rd,} Rn, Rm	Signed dual multiply add	Q
SMULBB, SMULBT, SMULTB, SMULTT	{Rd,} Rn, Rm	Signed multiply halfwords	-
SMULL	RdLo, RdHi, Rn, Rm	Signed multiply (32x32), 64-bit result	-
SMULWB, SMULWT	{Rd,} Rn, Rm	Signed multiply by halfword	-
SMUSD, SMUSDX	{Rd,} Rn, Rm	Signed dual multiply subtract	-
SSAT	Rd, #n, Rm {,shift #s}	Signed saturate	Q
SSAT16	Rd, #n, Rm	Signed saturate 16	Q
SSAX	{Rd,} Rn, Rm	Saturating subtract and add with exchange	GE

Mnemonic	Operands	Brief Description	Flags
SSUB16	{Rd,} Rn, Rm	Signed subtract 16	-
SSUB8	{Rd,} Rn, Rm	Signed subtract 8	-
STM	Rn{!}, reglist	Store multiple registers, increment after	-
STMDB, STMEA	Rn{!}, reglist	Store multiple registers, decrement before	-
STMFD, STMIA	Rn{!}, reglist	Store multiple registers, increment after	-
STR	Rt, [Rn {, #offset}]	Store register word	-
STRB, STRBT	Rt, [Rn {, #offset}]	Store register byte	-
STRD	Rt, Rt2, [Rn {, #offset}]	Store register two words	-
STREX	Rt, Rt, [Rn {, #offset}]	Store register exclusive	-
STREXB	Rd, Rt, [Rn]	Store register exclusive byte	-
STREXH	Rd, Rt, [Rn]	Store register exclusive halfword	-
STRH, STRHT	Rt, [Rn {, #offset}]	Store register halfword	-
STRSB, STRSBT	Rt, [Rn {, #offset}]	Store register signed byte	-
STRSH, STRSHT	Rt, [Rn {, #offset}]	Store register signed halfword	-
STRT	Rt, [Rn {, #offset}]	Store register word	-
SUB, SUBS	{Rd,} Rn, Op2	Subtract	N, Z, C, V
SUB, SUBW	{Rd,} Rn, #imm12	Subtract 12-bit constant	N, Z, C, V
SVC	#imm	Supervisor call	-
SXTAB	{Rd,} Rn, Rm, {,ROR #}	Extend 8 bits to 32 and add	-
SXTAB16	{Rd,} Rn, Rm, {,ROR #}	Dual extend 8 bits to 16 and add	-
SXTAH	{Rd,} Rn, Rm, {,ROR #}	Extend 16 bits to 32 and add	-
SXTB16	{Rd,} Rm {,ROR #n}	Signed extend byte 16	-
SXTB	{Rd,} Rm {,ROR #n}	Sign extend a byte	-
SXTH	{Rd,} Rm {,ROR #n}	Sign extend a halfword	-
TBB	[Rn, Rm]	Table branch byte	-
TBH	[Rn, Rm, LSL #1]	Table branch halfword	-
TEQ	Rn, Op2	Test equivalence	N, Z, C
TST	Rn, Op2	Test	N, Z, C
UADD16	{Rd,} Rn, Rm	Unsigned add 16	GE
UADD8	{Rd,} Rn, Rm	Unsigned add 8	GE
UASX	{Rd,} Rn, Rm	Unsigned add and subtract with exchange	GE
UHADD16	{Rd,} Rn, Rm	Unsigned halving add 16	-
UHADD8	{Rd,} Rn, Rm	Unsigned halving add 8	-
UHASX	{Rd,} Rn, Rm	Unsigned halving add and subtract with exchange	-
UHSAX	{Rd,} Rn, Rm	Unsigned halving subtract and add with exchange	-
UHSUB16	{Rd,} Rn, Rm	Unsigned halving subtract 16	-
UHSUB8	{Rd,} Rn, Rm	Unsigned halving subtract 8	-
UBFX	Rd, Rn, #lsb, #width	Unsigned bit field extract	-
UDIV	{Rd,} Rn, Rm	Unsigned divide	-
UMAAL	RdLo, RdHi, Rn, Rm	Unsigned multiply accumulate accumulate long (32x32+64), 64-bit result	-
UMLAL	RdLo, RdHi, Rn, Rm	Unsigned multiply with accumulate (32x32+32+32), 64-bit result	-
UMULL	RdLo, RdHi, Rn, Rm	Unsigned multiply (32x 2), 64-bit result	-
UQADD16	{Rd,} Rn, Rm	Unsigned Saturating Add 16	-
UQADD8	{Rd,} Rn, Rm	Unsigned Saturating Add 8	-
UQASX	{Rd,} Rn, Rm	Unsigned Saturating Add and Subtract with Exchange	-
UQSAX	{Rd,} Rn, Rm	Unsigned Saturating Subtract and Add with Exchange	-
UQSUB16	{Rd,} Rn, Rm	Unsigned Saturating Subtract 16	-
UQSUB8	{Rd,} Rn, Rm	Unsigned Saturating Subtract 8	-
USAD8	{Rd,} Rn, Rm	Unsigned Sum of Absolute Differences	-
USADA8	{Rd,} Rn, Rm, Ra	Unsigned Sum of Absolute Differences and Accumulate	-
USAT	Rd, #n, Rm {,shift #s}	Unsigned Saturate	Q
USAT16	Rd, #n, Rm	Unsigned Saturate 16	Q
USAX	{Rd,} Rn, Rm	Unsigned Subtract and add with Exchange	GE
USUB16	{Rd,} Rn, Rm	Unsigned Subtract 16	GE
USUB8	{Rd,} Rn, Rm	Unsigned Subtract 8	GE
XTAB	{Rd,} Rn, Rm, {,ROR #}	Rotate, extend 8 bits to 32 and Add	-
XTAB16	{Rd,} Rn, Rm, {,ROR #}	Rotate, dual extend 8 bits to 16 and Add	-
XTAH	{Rd,} Rn, Rm, {,ROR #}	Rotate, unsigned extend and Add Halfword	-
XTB	{Rd,} Rm, {,ROR #n}	Zero extend a Byte	-
XTB16	{Rd,} Rm, {,ROR #n}	Unsigned Extend Byte 16	-
UXTH	{Rd,} Rm, {,ROR #n}	Zero extend a Halfword	-
VABS.F32	Sd, Sm	Floating-point Absolute	-
VADD.F32	{Sd,} Sn, Sm	Floating-point Add	-
VCMP.F32	Sd, <Sm #0.0>	Compare two floating-point registers, or one floating-point register and zero	FPSCR

Mnemonic	Operands	Brief Description	Flags
VCMPE.F32	Sd, <Sm #0.0>	Compare two floating-point registers, or one floating-point register and zero with Invalid Operation check	FPSCR
VCVT.S32.F32	Sd, Sm	Convert between floating-point and integer	-
VCVT.S16.F32	Sd, Sd, #fbits	Convert between floating-point and fixed point	-
VCVTR.S32.F32	Sd, Sm	Convert between floating-point and integer with rounding	-
VCVT<B H>.F32.F16	Sd, Sm	Converts half-precision value to single-precision	-
VCVTT<B T>.F32.F16	Sd, Sm	Converts single-precision register to half-precision	-
VDIV.F32	{Sd,} Sn, Sm	Floating-point Divide	-
VFMA.F32	{Sd,} Sn, Sm	Floating-point Fused Multiply Accumulate	-
VFNMA.F32	{Sd,} Sn, Sm	Floating-point Fused Negate Multiply Accumulate	-
VFMS.F32	{Sd,} Sn, Sm	Floating-point Fused Multiply Subtract	-
VFNMS.F32	{Sd,} Sn, Sm	Floating-point Fused Negate Multiply Subtract	-
VLDM.F<32 64>	Rn{!}, list	Load Multiple extension registers	-
VLDR.F<32 64>	<Dd Sd>, [Rn]	Load an extension register from memory	-
VLMA.F32	{Sd,} Sn, Sm	Floating-point Multiply Accumulate	-
VLMS.F32	{Sd,} Sn, Sm	Floating-point Multiply Subtract	-
VMOV.F32	Sd, #imm	Floating-point Move immediate	-
VMOV	Sd, Sm	Floating-point Move register	-
VMOV	Sn, Rt	Copy ARM core register to single precision	-
VMOV	Sm, Sm1, Rt, Rt2	Copy 2 ARM core registers to 2 single precision	-
VMOV	Dd[x] , Rt	Copy ARM core register to scalar	-
VMOV	Rt, Dn[x]	Copy scalar to ARM core register	-
VMRS	Rt, FPSCR	Move FPSCR to ARM core register or APSR	N, Z, C, V
VMSR	FPSCR, Rt	Move to FPSCR from ARM Core register	FPSCR
VMUL.F32	{Sd,} Sn, Sm	Floating-point Multiply	-
VNEG.F32	Sd, Sm	Floating-point Negate	-
VNMLA.F32	{Sd,} Sn, Sm	Floating-point Multiply and Add	-
VNMLS.F32	{Sd,} Sn, Sm	Floating-point Multiply and Subtract	-
VNMUL	{Sd,} Sn, Sm	Floating-point Multiply	-
VPOP	list	Pop extension registers	-
VPUSH	list	Push extension registers	-
VSQRT.F32	Sd, Sm	Calculates floating-point Square Root	-
VSTM	Rn{!}, list	Floating-point register Store Multiple	-
VSTR.F<32 64>	Sd, [Rn]	Stores an extension register to memory	-
VSUB.F<32 64>	{Sd,} Sn, Sm	Floating-point Subtract	-
WFE	-	Wait for event	-
WFI	-	Wait for interrupt	-

Core Peripheral Register Regions

Address	Core Peripheral	Description (see page ...)
0xE000.E010-0xE000.E01F	System Timer	123
0xE000.E100-0xE000.E4EF 0xE000.EF00-0xE000.EF03	Nested Vectored Interrupt Controller	124
0xE000.E008-0xE000.E00F 0xE000.ED00-0xE000.ED3F	System Control Block	125
0xE000.ED90-0xE000.EDB8	Memory Protection Unit	125
0xE000.EF30-0xE000.EF44	Floating Point Unit	130

Processor Core

- 32-bit ARM Cortex-M4F architecture optimized for small-footprint embedded applications
- 80-MHz operation; 100 DMIPS performance
- Thumb-2 mixed 16-/32-bit instruction set delivers the high performance expected of a 32-bit
- ARM core in a compact memory size usually associated with 8- and 16-bit devices, typically in
- the range of a few kilobytes of memory for microcontroller-class applications
 - Single-cycle multiply instruction and hardware divide
 - Atomic bit manipulation (bit-banding), delivering maximum memory utilization and streamlined peripheral control
 - Unaligned data access, enabling data to be efficiently packed into memory
- IEEE754-compliant single-precision Floating-Point Unit (FPU)
- 16-bit SIMD vector processing unit
- Fast code execution permits slower processor clock or increases sleep mode time
- **Harvard architecture** characterized by separate buses for instruction and data
- Efficient processor core, system and memories
- Hardware division and fast digital-signal-processing orientated multiply accumulate
- Saturating arithmetic for signal processing
- Deterministic, high-performance interrupt handling for time-critical applications
- Memory protection unit (MPU) to provide a privileged mode for protected operating system functionality
- Enhanced system debug with extensive breakpoint and **trace capabilities** **Serial Wire Debug** and **Serial Wire Trace** **reduce the number of pins required for debugging and tracing**

- Migration from the ARM7™ processor family for better performance and power efficiency
- Optimized for single-cycle Flash memory usage up to specific frequencies; see “Internal Memory” on page 524 for more information.
- Ultra-low power consumption with integrated sleep modes

System Timer

ARM Cortex-M4F includes an integrated system timer, SysTick. SysTick provides a simple, 24-bit, clear-on-write, decrementing, wrap-on-zero counter with a flexible control mechanism. The counter can be used in several different ways

- An RTOS tick timer that fires at a programmable rate (for example, 100 Hz) and invokes a SysTick routine
- A high-speed alarm timer using the system clock
- A variable rate alarm or signal timer—the duration is range-dependent on the reference clock used and the dynamic range of the counter
- A simple counter used to measure time to completion and time used
- An internal clock-source control based on missing/meeting durations

Nested Vectored Interrupt Controller (NVIC) (see page 124)

The TM4C123GH6PM controller includes the ARM Nested Vectored Interrupt Controller (NVIC). The NVIC and Cortex-M4F prioritize and handle all exceptions in Handler Mode. The processor state is automatically stored to the stack on an exception and automatically restored from the stack at the end of the Interrupt Service Routine (ISR). The interrupt vector is fetched in parallel to the state saving, enabling efficient interrupt entry. The processor supports tail-chaining, meaning that back-to-back interrupts can be performed without the overhead of state saving and restoration. Software can set eight priority levels on 7 exceptions (system handlers) and 78 interrupts.

System Control Block (SCB) (see page 125)

The SCB provides system implementation information and system control, including configuration, control, and reporting of system exceptions.

Memory Protection Unit (MPU) (see page 125)

The MPU supports the standard ARM7 Protected Memory System Architecture (PMSA) model. The MPU provides full support for protection regions, overlapping protection regions, access permissions, and exporting memory attributes to the system.

Floating-Point Unit (FPU) (see page 130)

The FPU fully supports single-precision add, subtract, multiply, divide, multiply and accumulate, and square root operations. It also provides conversions between fixed-point and floating-point data formats, and floating-point constant instructions.

- 32-bit instructions for single-precision (C float) data-processing operations
- Combined multiply and accumulate instructions for increased precision (Fused MAC)
- Hardware support for conversion, addition, subtraction, multiplication with optional accumulate, division, and square-root
- Hardware support for denormals and all IEEE rounding modes
- 32 dedicated 32-bit single-precision registers, also addressable as 16 double-word registers
- Decoupled three stage pipeline

On-Chip Memory

The TM4C123GH6PM microcontroller is integrated with the following set of on-chip memory and features:

- 32 KB single-cycle SRAM
- 256 KB Flash memor
- 2KB EEPROM
- Internal ROM loaded with TivaWare™ for C Series software:
 - TivaWare™ Peripheral Driver Library
 - TivaWare Boot Loader
 - Advanced Encryption Standard (AES) cryptography tables
 - Cyclic Redundancy Check (CRC) error detection functionality

SRAM (see page 525)

The TM4C123GH6PM microcontroller provides 32 KB of single-cycle on-chip SRAM. The internal SRAM of the device is located at offset 0x2000.0000 of the device memory map. Because read-modify-write (RMW) operations are very time consuming, ARM has introduced bit-banding technology in the Cortex-M4F processor. With a bit-band-enabled processor, certain regions in the memory map (SRAM and peripheral space) can use address aliases to access individual bits in a single, atomic operation.

Data can be transferred to and from SRAM by the following masters:

- μ DMA
- USB

Flash Memory (see page 528)

The TM4C123GH6PM microcontroller provides 256 KB of single-cycle on-chip Flash memory. The Flash memory is organized as a set of 1-KB blocks that can be individually erased. Erasing a block causes the entire contents of the block to be reset to all 1s. These blocks are paired into a set of 2-KB blocks that can be individually protected. The blocks can be marked as read-only or execute-only, providing different levels of code protection. Read-only blocks cannot be erased or programmed, protecting the contents of those blocks from being modified. Execute-only blocks cannot be erased or programmed, and can only be read by the controller instruction fetch mechanism, protecting the contents of those blocks from being read by either the controller or by a debugger.

ROM (see page 526)

The TM4C123GH6PM ROM is preprogrammed with the following software and programs:

- TivaWare Peripheral Driver Library
- TivaWare Boot Loader
- Advanced Encryption Standard (AES) cryptography tables
- Cyclic Redundancy Check (CRC) error-detection functionality

The TivaWare Peripheral Driver Library is a royalty-free software library for controlling on-chip peripherals with a boot-loader capability. The library performs both peripheral initialization and control functions, with a choice of polled or interrupt-driven peripheral support. In addition, the library is designed to take full advantage of the stellar interrupt performance of the ARM Cortex-M4F core. No special pragmas or custom assembly code prologue/epilogue functions are required. For applications that require in-field programmability, the royalty-free TivaWare Boot Loader can act as an application loader and support in-field firmware updates.

The Advanced Encryption Standard (AES) is a publicly defined encryption standard used by the U.S. Government. AES is a strong encryption method with reasonable performance and size. In addition, it is fast in both hardware and software, is fairly easy to implement, and requires little memory. The Texas Instruments encryption package

is available with full source code, and is based on Lesser General Public License (LGPL) source. An LGPL means that the code can be used within an application without any copyleft implications for the application (the code does not automatically become open source). Modifications to the package source, however, must be open source.

CRC (Cyclic Redundancy Check) is a technique to validate a span of data has the same contents as when previously checked. This technique can be used to validate correct receipt of messages (nothing lost or modified in transit), to validate data after decompression, to validate that Flash memory contents have not been changed, and for other cases where the data needs to be validated. A CRC is preferred over a simple checksum (for example, XOR all bits) because it catches changes more readily.

EEPROM (see page 534)

The TM4C123GH6PM microcontroller includes an EEPROM with the following features:

- 2Kbytes of memory accessible as 512 32-bit words
- 32 blocks of 16 words (64 bytes) each
- Built-in wear leveling
- Access protection per block
- Lock protection option for the whole peripheral as well as per block using 32-bit to 96-bit unlock codes (application selectable)

Serial Communications Peripherals

The TM4C123GH6PM controller supports both asynchronous and synchronous serial communications with:

- Two CAN 2.0 A/B controllers
- USB 2.0 OTG/Host/Device
- Eight UARTs with IrDA, 9-bit and ISO 7816 support.
- Four I2C modules with four transmission speeds including high-speed mode
- Four Synchronous Serial Interface modules (SSI)

The following sections provide more detail on each of these communications functions.

Controller Area Network (CAN) (see page 1048)

Controller Area Network (CAN) is a multicast shared serial-bus standard for connecting electronic control units (ECUs). CAN was specifically designed to be robust in electromagnetically noisy environments and can utilize a differential balanced line like RS-485 or twisted-pair wire. Originally created for automotive purposes, it is now used in many embedded control applications (for example, industrial or medical). Bit rates up to 1 Mbps are possible at network lengths below 40 meters. Decreased bit rates allow longer network distances (for example, 125 Kbps at 500m).

A transmitter sends a message to all CAN nodes (broadcasting). Each node decides on the basis of the identifier received whether it should process the message. The identifier also determines the priority that the message enjoys in competition for bus access. Each CAN message can transmit from 0 to 8 bytes of user information.

The TM4C123GH6PM microcontroller includes two CAN units with the following features:

- CAN protocol version 2.0 part A/B
- Bit rates up to 1 Mbps
- 32 message objects with individual identifier masks
- Maskable interrupt
- Disable Automatic Retransmission mode for Time-Triggered CAN (TTCAN) applications
- Programmable loopback mode for self-test operation
- Programmable FIFO mode enables storage of multiple message objects
- Gluelessly attaches to an external CAN transceiver through the CANnTX and CANnRX signals

Universal Serial Bus (USB) (see page 1099)

Universal Serial Bus (USB) is a serial bus standard designed to allow peripherals to be connected and disconnected using a standardized interface without rebooting the system. The TM4C123GH6PM microcontroller supports three configurations in USB 2.0 full and low speed:

USB Device, USB Host, and USB On-The-Go (negotiated on-the-go as host or device when connected to other USB-enabled systems).

The USB module has the following features:

- Complies with USB-IF (Implementer's Forum) certification standards
- USB 2.0 full-speed (12 Mbps) and low-speed (1.5 Mbps) operation with integrated PHY
- 4 transfer types: Control, Interrupt, Bulk, and Isochronous
- 16 endpoints
 - 1 dedicated control IN endpoint and 1 dedicated control OUT endpoint
 - 7 configurable IN endpoints and 7 configurable OUT endpoints
- 4 KB dedicated endpoint memory: one endpoint may be defined for double-buffered 1023-byte isochronous packet size
- VBUS droop and valid ID detection and interrupt
- Efficient transfers using Micro Direct Memory Access Controller (μDMA)
 - Separate channels for transmit and receive for up to three IN endpoints and three OUT endpoints
 - Channel requests asserted when FIFO contains required amount of data

UART (see page 893)

A Universal Asynchronous Receiver/Transmitter (UART) is an integrated circuit used for RS-232C serial communications, containing a transmitter (parallel-to-serial converter) and a receiver (serial-to-parallel converter), each clocked separately.

The TM4C123GH6PM microcontroller includes eight fully programmable 16C550-type UARTs. Although the functionality is similar to a 16C550 UART, this UART design is not register compatible.

The UART can generate individually masked interrupts from the Rx, Tx, modem flow control, and error conditions. The module generates a single combined interrupt when any of the interrupts are asserted and are unmasked.

The eight UARTs have the following features:

- Programmable baud-rate generator allowing speeds up to 5 Mbps for regular speed (divide by 16) and 10 Mbps for high speed (divide by 8)
- Separate 16x8 transmit (TX) and receive (RX) FIFOs to reduce CPU interrupt service loading
- Programmable FIFO length, including 1-byte deep operation providing conventional double-buffered interface
- FIFO trigger levels of 1/8, 1/4, 1/2, 3/4, and 7/8
- Standard asynchronous communication bits for start, stop, and parity
- Line-break generation and detection
- Fully programmable serial interface characteristics
 - 5, 6, 7, or 8 data bits
 - Even, odd, stick, or no-parity bit generation/detection
 - 1 or 2 stop bit generation
- IrDA serial-IR (SIR) encoder/decoder providing
 - Programmable use of IrDA Serial Infrared (SIR) or UART input/output
 - Support of IrDA SIR encoder/decoder functions for data rates up to 115.2 Kbps half-duplex

- Support of normal 3/16 and low-power (1.41-2.23 μ s) bit durations
- Programmable internal clock generator enabling division of reference clock by 1 to 256 for low-power mode bit duration
- Support for communication with ISO 7816 smart cards
- Modem flow control (on UART1)
- EIA-485 9-bit support
- Standard FIFO-level and End-of-Transmission interrupts
- Efficient transfers using Micro Direct Memory Access Controller (μ DMA)
- Separate channels for transmit and receive
- Receive single request asserted when data is in the FIFO; burst request asserted at programmed FIFO level
- Transmit single request asserted when there is space in the FIFO; burst request asserted at programmed FIFO level

I2C (see page 997)

The Inter-Integrated Circuit (I2C) bus provides bi-directional data transfer through a two-wire design (a serial data line SDA and a serial clock line SCL). The I2C bus interfaces to external I2C devices such as serial memory (RAMs and ROMs), networking devices, LCDs, tone generators, and so on.

The I2C bus may also be used for system testing and diagnostic purposes in product development and manufacture.

Each device on the I2C bus can be designated as either a master or a slave. I2C module supports both sending and receiving data as either a master or a slave and can operate simultaneously as both a master and a slave. Both the I2C master and slave can generate interrupts.

The TM4C123GH6PM microcontroller includes four I2C modules with the following features:

- Devices on the I2C bus can be designated as either a master or a slave
- Supports both transmitting and receiving data as either a master or a slave
- Supports simultaneous master and slave operation
- Four I2C modes
- Master transmit
- Master receive
- Slave transmit
- Slave receive
- Four transmission speeds:
- Standard (100 Kbps)
- Fast-mode (400 Kbps)
- Fast-mode plus (1 Mbps)
- High-speed mode (3.33 Mbps)
- Clock low timeout interrupt
- Dual slave address capability
- Glitch suppression
- Master and slave interrupt generation
- Master generates interrupts when a transmit or receive operation completes (or aborts due to an error)
- Slave generates interrupts when data has been transferred or requested by a master or when a START or STOP condition is detected
- Master with arbitration and clock synchronization, multimaster support, and 7-bit addressing mode

SSI (see page 952)

Synchronous Serial Interface (SSI) is a four-wire bi-directional communications interface that converts data between parallel and serial. The SSI module performs serial-to-parallel conversion on data received from a peripheral device, and parallel-to-serial conversion on data transmitted to a peripheral device. The SSI module can be configured as either a master or slave device. As a slave device, the SSI module can also be configured to disable its output, which allows a master device to be coupled with multiple slave devices. The TX and RX paths are buffered with separate internal FIFOs.

The SSI module also includes a programmable bit rate clock divider and prescaler to generate the output serial clock derived from the SSI module's input clock. Bit rates are generated based on the input clock and the maximum bit rate is determined by the connected peripheral.

The TM4C123GH6PM microcontroller includes four SSI modules with the following features:

- Programmable interface operation for Freescale SPI, MICROWIRE, or Texas Instruments synchronous serial interfaces
- Master or slave operation
- Programmable clock bit rate and prescaler
- Separate transmit and receive FIFOs, each 16 bits wide and 8 locations deep
- Programmable data frame size from 4 to 16 bits
- Internal loopback test mode for diagnostic/debug testing
- Standard FIFO-based interrupts and End-of-Transmission interrupt
- Efficient transfers using Micro Direct Memory Access Controller (μ DMA)
- Separate channels for transmit and receive
- Receive single request asserted when data is in the FIFO; burst request asserted when FIFO contains 4 entries
- Transmit single request asserted when there is space in the FIFO; burst request asserted when four or more entries are available to be written in the FIFO

Direct Memory Access (see page 585)

The TM4C123GH6PM microcontroller includes a Direct Memory Access (DMA) controller, known as micro-DMA (μ DMA). The μ DMA controller provides a way to offload data transfer tasks from the Cortex-M4F processor, allowing for more efficient use of the processor and the available bus bandwidth. The μ DMA controller can perform transfers between memory and peripherals. It has dedicated channels for each supported on-chip module and can be programmed to automatically perform transfers between peripherals and memory as the peripheral is ready to transfer more data.

The μ DMA controller provides the following features:

- ARM PrimeCell 32-channel configurable μ DMA controller
- Support for memory-to-memory, memory-to-peripheral, and peripheral-to-memory in multiple transfer modes
- Basic for simple transfer scenarios
- Ping-pong for continuous data flow
- Scatter-gather for a programmable list of up to 256 arbitrary transfers initiated from a single request
- Highly flexible and configurable channel operation
- Independently configured and operated channels
- Dedicated channels for supported on-chip modules
- Flexible channel assignments
- One channel each for receive and transmit path for bidirectional modules
- Dedicated channel for software-initiated transfers
- Per-channel configurable priority scheme
- Optional software-initiated requests for any channel
- Two levels of priority
- Design optimizations for improved bus access performance between μ DMA controller and the processor core
- μ DMA controller access is subordinate to core access
- RAM striping

- Peripheral bus segmentation
- Data sizes of 8, 16, and 32 bits
- Transfer size is programmable in binary steps from 1 to 1024
- Source and destination address increment size of byte, half-word, word, or no increment
- Maskable peripheral requests
- Interrupt on transfer completion, with a separate interrupt per channel

System Control and Clocks (see page 212)

System control determines the overall operation of the device. It provides information about the device, controls power-saving features, controls the clocking of the device and individual peripherals, and handles reset detection and reporting.

- Device identification information: version, part number, SRAM size, Flash memory size, and so on
- Power control
 - On-chip fixed Low Drop-Out (LDO) voltage regulator
 - Hibernation module handles the power-up/down 3.3 V sequencing and control for the core digital logic and analog circuits
 - Low-power options for microcontroller: Sleep and Deep-Sleep modes with clock gating
 - Low-power options for on-chip modules: software controls shutdown of individual peripherals and memory
 - 3.3-V supply brown-out detection and reporting via interrupt or reset
- Multiple clock sources for microcontroller system clock. The following clock sources are provided to the TM4C123GH6PM microcontroller:
 - Precision Internal Oscillator (PIOSC) providing a 16-MHz frequency
 - 16 MHz $\pm 3\%$ across temperature and voltage
 - Can be recalibrated with 7-bit trim resolution to achieve better accuracy (16 MHz $\pm 1\%$)
 - Software power down control for low power modes
- Main Oscillator (MOSC): A frequency-accurate clock source by one of two means: an external single-ended clock source is connected to the OSC0 input pin, or an external crystal is connected across the OSC0 input and OSC1 output pins.
- Low Frequency Internal Oscillator (LFIOSC): On-chip resource used during power-saving modes
- Hibernate RTC oscillator (RTCOSC) clock that can be configured to be the 32.768-kHz external oscillator source from the Hibernation (HIB) module or the HIB Low Frequency clock source (HIB LFIOSC), which is located within the Hibernation Module.
- Flexible reset sources
 - Power-on reset (POR)
 - Reset pin assertion
 - Brown-out reset (BOR) detector alerts to system power drops
 - Software reset
 - Watchdog timer reset
 - MOSC failure

Programmable Timers (see page 704)

Programmable timers can be used to count or time external events that drive the Timer input pins. Each 16/32-bit GPTM block provides two 16-bit timers/counters that can be configured to operate independently as timers or event counters, or configured to operate as one 32-bit timer or one 32-bit Real-Time Clock (RTC). Each 32/64-bit Wide GPTM block provides two 32-bit timers/counters that can be configured to operate independently as timers or event counters, or configured to operate as one 64-bit timer or one 64-bit Real-Time Clock (RTC). Timers can also be used to trigger analog-to-digital (ADC) conversions and DMA transfers.

The General-Purpose Timer Module (GPTM) contains six 16/32-bit GPTM blocks and six 32/64-bit Wide GPTM blocks with the following functional options:

- 16/32-bit operating modes:
 - 16- or 32-bit programmable one-shot timer
 - 16- or 32-bit programmable periodic timer
 - 16-bit general-purpose timer with an 8-bit prescaler
 - 32-bit Real-Time Clock (RTC) when using an external 32.768-KHz clock as the input
 - 16-bit input-edge count- or time-capture modes with an 8-bit prescaler
 - 16-bit PWM mode with an 8-bit prescaler and software-programmable output inversion of the PWM signal
- 32/64-bit operating modes:
 - 32- or 64-bit programmable one-shot timer
 - 32- or 64-bit programmable periodic timer
 - 32-bit general-purpose timer with a 16-bit prescaler
 - 64-bit Real-Time Clock (RTC) when using an external 32.768-KHz clock as the input
 - 32-bit input-edge count- or time-capture modes with a 16-bit prescaler
 - 32-bit PWM mode with a 16-bit prescaler and software-programmable output inversion of the PWM signal
- Count up or down
- Twelve 16/32-bit Capture Compare PWM pins (CCP)
- Twelve 32/64-bit Capture Compare PWM pins (CCP)
- Daisy chaining of timer modules to allow a single timer to initiate multiple timing events
- Timer synchronization allows selected timers to start counting on the same clock cycle
- ADC event trigger
- User-enabled stalling when the microcontroller asserts CPU Halt flag during debug (excluding RTC mode)
- Ability to determine the elapsed time between the assertion of the timer interrupt and entry into the interrupt service routine
- Efficient transfers using Micro Direct Memory Access Controller (μ DMA)
- Dedicated channel for each timer
- Burst request generated on timer interrupt

CCP Pins (see page 712)

Capture Compare PWM pins (CCP) can be used by the General-Purpose Timer Module to time/count external events using the CCP pin as an input. Alternatively, the GPTM can generate a simple PWM output on the CCP pin.

The TM4C123GH6PM microcontroller includes twelve 16/32-bit CCP pins that can be programmed to operate in the following modes:

- Capture: The GP Timer is incremented/decremented by programmed events on the CCP input. The GP Timer captures and stores the current timer value when a programmed event occurs.
- Compare: The GP Timer is incremented/decremented by programmed events on the CCP input. The GP Timer compares the current value with a stored value and generates an interrupt when a match occurs.
- PWM: The GP Timer is incremented/decremented by the system clock. A PWM signal is generated based on a match between the counter value and a value stored in a match register and is output on the CCP pin.

Hibernation Module (HIB) (see page 493)

The Hibernation module provides logic to switch power off to the main processor and peripherals and to wake on external or time-based events. The Hibernation module includes power-sequencing logic and has the following features:

- 32-bit real-time seconds counter (RTC) with 1/32,768 second resolution and a 15-bit sub-seconds counter
- 32-bit RTC seconds match register and a 15-bit sub seconds match for timed wake-up and interrupt generation with 1/32,768 second resolution
- RTC predivider trim for making fine adjustments to the clock rate

- Two mechanisms for power control
 - System power control using discrete external regulator
 - On-chip power control using internal switches under register control
- Dedicated pin for waking using an external signal
- RTC operational and hibernation memory valid as long as VDD or VBAT is valid
- Low-battery detection, signaling, and interrupt generation, with optional wake on low battery
- GPIO pin state can be retained during hibernation
- Clock source from a 32.768-kHz external crystal or oscillator
- Sixteen 32-bit words of battery-backed memory to save state during hibernation
- Programmable interrupts for:
 - RTC match
 - External wake
 - Low battery

Watchdog Timers (see page 774)

A watchdog timer is used to regain control when a system has failed due to a software error or to the failure of an external device to respond in the expected way. The TM4C123GH6PM Watchdog Timer can generate an interrupt, a non-maskable interrupt, or a reset when a time-out value is reached. In addition, the Watchdog Timer is ARM FIRM-compliant and can be configured to generate an interrupt to the microcontroller on its first time-out, and to generate a reset signal on its second timeout. Once the Watchdog Timer has been configured, the lock register can be written to prevent the timer configuration from being inadvertently altered. The TM4C123GH6PM microcontroller has two Watchdog Timer modules: Watchdog Timer 0 uses the system clock for its timer clock; Watchdog Timer 1 uses the PLOSC as its timer clock. The Watchdog Timer module has the following features:

- 32-bit down counter with a programmable load register
- Separate watchdog clock with an enable
- Programmable interrupt generation logic with interrupt masking and optional NMI function
- Lock register protection from runaway software
- Reset generation logic with an enable/disable
- User-enabled stalling when the microcontroller asserts the CPU Halt flag during debug

Programmable GPIOs (see page 649)

General-purpose input/output (GPIO) pins offer flexibility for a variety of connections. The TM4C123GH6PM GPIO module is comprised of six physical GPIO blocks, each corresponding to an individual GPIO port. The GPIO module is FIRM-compliant (compliant to the ARM Foundation IP for Real-Time Microcontrollers specification) and supports 0-43 programmable input/output pins. The number of GPIOs available depends on the peripherals being used (see “Signal Tables” on page 1329 for the signals available to each GPIO pin).

- Up to 43 GPIOs, depending on configuration
- Highly flexible pin muxing allows use as GPIO or one of several peripheral functions
- 5-V-tolerant in input configuration
- Ports A-G accessed through the Advanced Peripheral Bus (APB)
- Fast toggle capable of a change every clock cycle for ports on AHB, every two clock cycles for ports on APB
- Programmable control for GPIO interrupts
 - Interrupt generation masking
 - Edge-triggered on rising, falling, or both
 - Level-sensitive on High or Low values
- Bit masking in both read and write operations through address lines
- Can be used to initiate an ADC sample sequence or a μ DMA transfer
- Pin state can be retained during Hibernation mode
- Pins configured as digital inputs are Schmitt-triggered
- Programmable control for GPIO pad configuration
 - Weak pull-up or pull-down resistors
 - 2-mA, 4-mA, and 8-mA pad drive for digital communication; up to four pads can sink 18-mA for high-current applications
 - Slew rate control for 8-mA pad drive
 - Open drain enables
 - Digital input enables

PWM (see page 1230)

The TM4C123GH6PM microcontroller contains two PWM modules, each with four PWM generator blocks and a control block, for a total of 16 PWM outputs. Pulse width modulation (PWM) is a powerful technique for digitally encoding analog signal levels. High-resolution counters are used to generate a square wave, and the duty cycle of the square wave is modulated to encode an analog signal. Typical applications include switching power supplies and motor control. Each TM4C123GH6PM PWM module consists of four PWM generator block and a control block. Each PWM generator block contains one timer (16-bit down or up/down counter), two comparators, a PWM signal generator, a dead-band generator, and an interrupt/ADC-trigger selector. Each PWM generator block produces two PWM signals that can either be independent signals or a single pair of complementary signals with dead-band delays inserted.

Each PWM generator has the following features:

- One fault-condition handling inputs to quickly provide low-latency shutdown and prevent damage to the motor being controlled, for a total of two inputs
- One 16-bit counter
 - Runs in Down or Up/Down mode
 - Output frequency controlled by a 16-bit load value
 - Load value updates can be synchronized
 - Produces output signals at zero and load value
- Two PWM comparators
 - Comparator value updates can be synchronized
 - Produces output signals on match
- PWM signal generator
 - Output PWM signal is constructed based on actions taken as a result of the counter and PWM comparator output signals
 - Produces two independent PWM signals
- Dead-band generator
 - Produces two PWM signals with programmable dead-band delays suitable for driving a half-H bridge
 - Can be bypassed, leaving input PWM signals unmodified
- Can initiate an ADC sample sequence

The control block determines the polarity of the PWM signals and which signals are passed through to the pins. The output of the PWM generation blocks are managed by the output control block before being passed to the device pins. The PWM control block has the following options:

- PWM output enable of each PWM signal
- Optional output inversion of each PWM signal (polarity control)
- Optional fault handling for each PWM signal
- Synchronization of timers in the PWM generator blocks
- Synchronization of timer/comparator updates across the PWM generator blocks

- Extended PWM synchronization of timer/comparator updates across the PWM generator blocks
- Interrupt status summary of the PWM generator blocks
- Extended PWM fault handling, with multiple fault signals, programmable polarities, and filtering
- PWM generators can be operated independently or synchronized with other generators

Analog

The TM4C123GH6PM microcontroller provides analog functions integrated into the device, including:

- Two 12-bit Analog-to-Digital Converters (ADC), with a total of 12 analog input channels and each with a sample rate of one million samples/second
- Two analog comparators
- On-chip voltage regulator

The following provides more detail on these analog functions.

ADC (see page 799)

An analog-to-digital converter (ADC) is a peripheral that converts a continuous analog voltage to a discrete digital number. The TM4C123GH6PM ADC module features 12-bit conversion resolution and supports 12 input channels plus an internal temperature sensor. Four buffered sample sequencers allow rapid sampling of up to 12 analog input sources without controller intervention. Each sample sequencer provides flexible programming with fully configurable input source, trigger events, interrupt generation, and sequencer priority. Each ADC module has a digital comparator function that allows the conversion value to be diverted to a comparison unit that provides eight digital comparators.

The TM4C123GH6PM microcontroller provides two ADC modules, each with the following features:

- 12 shared analog input channels
- 12-bit precision ADC
- Single-ended and differential-input configurations
- On-chip internal temperature sensor
- Maximum sample rate of one million samples/second
- Optional phase shift in sample time programmable from 22.5° to 337.5°
- Four programmable sample conversion sequencers from one to eight entries long, with corresponding conversion result FIFOs
- Flexible trigger control
 - Controller (software)
 - Timers
 - Analog Comparators
 - PWM
 - GPIO
- Hardware averaging of up to 64 samples
- Eight digital comparators
- Power and ground for the analog circuitry is separate from the digital power and ground
- Efficient transfers using Micro Direct Memory Access Controller (μDMA)
 - Dedicated channel for each sample sequencer
 - ADC module uses burst requests for DMA

Analog Comparators (see page 1215)

An analog comparator is a peripheral that compares two analog voltages and provides a logical output that signals the comparison result. The TM4C123GH6PM microcontroller provides two independent integrated analog comparators that can be configured to drive an output or generate an interrupt or ADC event. The comparator can provide its output to a device pin, acting as a replacement for an analog comparator on the board, or it can be used to signal the application via interrupts or triggers to the ADC to cause it to start capturing a sample sequence. The interrupt generation and ADC triggering logic is separate. This means, for example, that an interrupt can be generated on a rising edge and the ADC triggered on a falling edge.

The TM4C123GH6PM microcontroller provides two independent integrated analog comparators with the following functions:

- Compare external pin input to external pin input or to internal programmable voltage reference
- Compare a test voltage against any one of the following voltages:
 - An individual external reference voltage
 - A shared single external reference voltage
 - A shared internal reference voltage

System integration

The TM4C123GH6PM microcontroller provides a variety of standard system functions integrated into the device, including:

- Direct Memory Access Controller (DMA)
- System control and clocks including on-chip precision 16-MHz oscillator
- Six 32-bit timers (up to twelve 16-bit)
- Six wide 64-bit timers (up to twelve 32-bit)
- Twelve 32/64-bit Capture Compare PWM (CCP) pins
- Lower-power battery-backed Hibernation module
- Real-Time Clock in Hibernation module
- Two Watchdog Timers
 - One timer runs off the main oscillator
 - One timer runs off the precision internal oscillator
- Up to 43 GPIOs, depending on configuration
 - Highly flexible pin muxing allows use as GPIO or one of several peripheral functions
 - Independently configurable to 2-, 4- or 8-mA drive capability
 - Up to 4 GPIOs can have 18-mA drive capability

Clock

- Multiple clock sources for microcontroller system clock. The following clock sources are provided to the TM4C123GH6PM microcontroller:
 - Precision Internal Oscillator (PIOSC) providing a 16-MHz frequency
 - 16 MHz ±3% across temperature and voltage
 - Can be recalibrated with 7-bit trim resolution to achieve better accuracy (16 MHz ±1%)
 - Software power down control for low power modes
 - Main Oscillator (MOSC): A frequency-accurate clock source by one of two means: an external single-ended clock source is connected to the OSC0 input pin, or an external crystal is connected across the OSC0 input and OSC1 output pins.
 - Low Frequency Internal Oscillator (LFIOSC): On-chip resource used during power-saving modes
 - Hibernation RTC oscillator (RTCOSC) clock that can be configured to be the 32.768-kHz external oscillator source from the Hibernation (HIB) module or the HIB Low Frequency clock source (HIB LFIOSC), which is located within the Hibernation Module.
- Flexible reset sources
 - Power-on reset (POR)
 - Reset pin assertion

- Brown-out reset (BOR) detector alerts to system power drops
- Software reset
- Watchdog timer reset

Programmable Timers (see page 704)

Programmable timers can be used to count or time external events that drive the Timer input pins. Each 16/32-bit GPTM block provides two 16-bit timers/counters that can be configured to operate independently as timers or event counters, or configured to operate as one 32-bit timer or one 32-bit Real-Time Clock (RTC). Each 32/64-bit Wide GPTM block provides two 32-bit timers/counters that can be configured to operate independently as timers or event counters, or configured to operate as one 64-bit timer or one 64-bit Real-Time Clock (RTC). Timers can also be used to trigger analog-to-digital (ADC) conversions and DMA transfers.

The General-Purpose Timer Module (GPTM) contains six 16/32-bit GPTM blocks and six 32/64-bit Wide GPTM blocks with the following functional options:

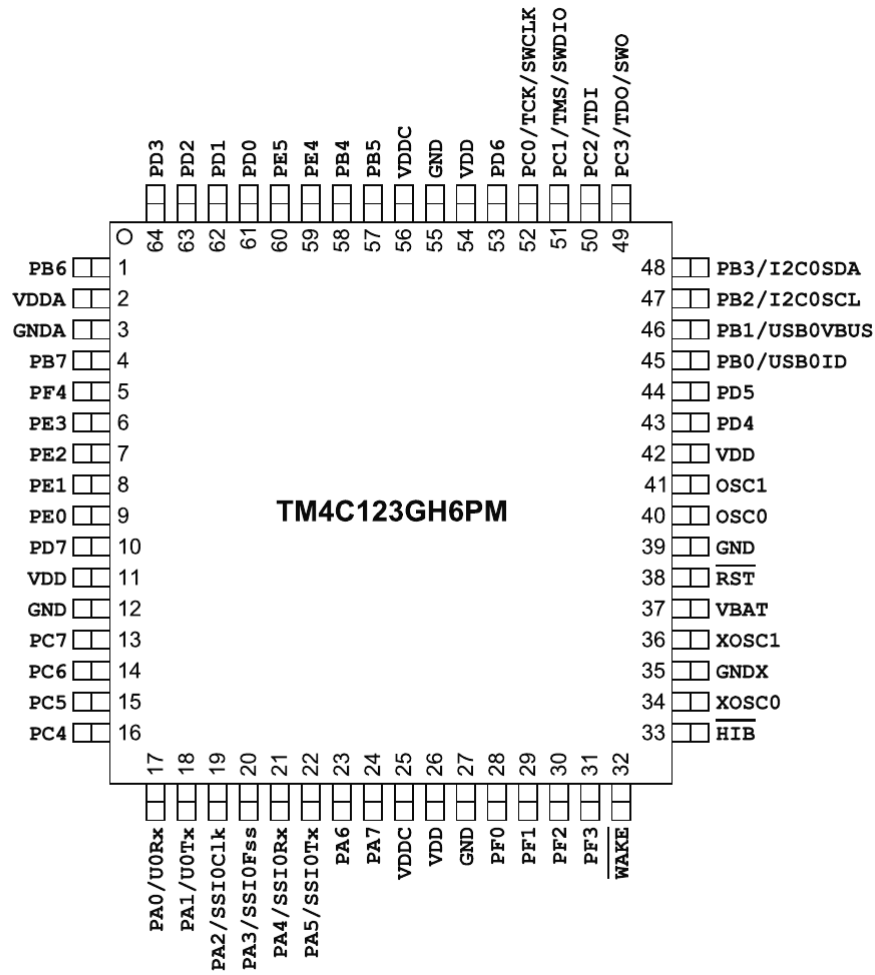
- 16/32-bit operating modes:
 - 16- or 32-bit programmable one-shot timer
 - 16- or 32-bit programmable periodic timer
 - 16-bit general-purpose timer with an 8-bit prescaler
 - 32-bit Real-Time Clock (RTC) when using an external 32.768-KHz clock as the input
 - 16-bit input-edge count- or time-capture modes with an 8-bit prescaler
 - 16-bit PWM mode with an 8-bit prescaler and software-programmable output inversion of the PWM signal
- 32/64-bit operating modes:
 - 32- or 64-bit programmable one-shot timer
 - 32- or 64-bit programmable periodic timer
 - 32-bit general-purpose timer with a 16-bit prescaler
 - 64-bit Real-Time Clock (RTC) when using an external 32.768-KHz clock as the input
 - 32-bit input-edge count- or time-capture modes with a 16-bit prescaler
 - 32-bit PWM mode with a 16-bit prescaler and software-programmable output inversion of the PWM signal
- Count up or down
- Twelve 16/32-bit Capture Compare PWM pins (CCP)
- Twelve 32/64-bit Capture Compare PWM pins (CCP)
- Daisy chaining of timer modules to allow a single timer to initiate multiple timing events
- Timer synchronization allows selected timers to start counting on the same clock cycle
- ADC event trigger
- User-enabled stalling when the microcontroller asserts CPU Halt flag during debug (excluding RTC mode)
- Ability to determine the elapsed time between the assertion of the timer interrupt and entry into the interrupt service routine
- Efficient transfers using Micro Direct Memory Access Controller (μDMA)
 - Dedicated channel for each timer
 - Burst request generated on timer interrupt

JTAG and ARM Serial Wire Debug (see page 200)

The Joint Test Action Group (JTAG) port is an IEEE standard that defines a Test Access Port and Boundary Scan Architecture for digital integrated circuits and provides a standardized serial interface for controlling the associated test logic. The TAP, Instruction Register (IR), and Data Registers (DR) can be used to test the interconnections of assembled printed circuit boards and obtain manufacturing information on the components. The JTAG Port also provides a means of accessing and controlling design-for-test features such as I/O pin observation and control, scan testing, and debugging. Texas Instruments replaces the ARM SW-DP and JTAG-DP with the ARM Serial Wire JTAG Debug Port (SWJ-DP) interface. The SWJ-DP interface combines the SWD and JTAG debug ports into one module providing all the normal JTAG debug and test functionality plus real-time access to system memory without halting the core or requiring any target resident code. The SWJ-DP interface has the following features:

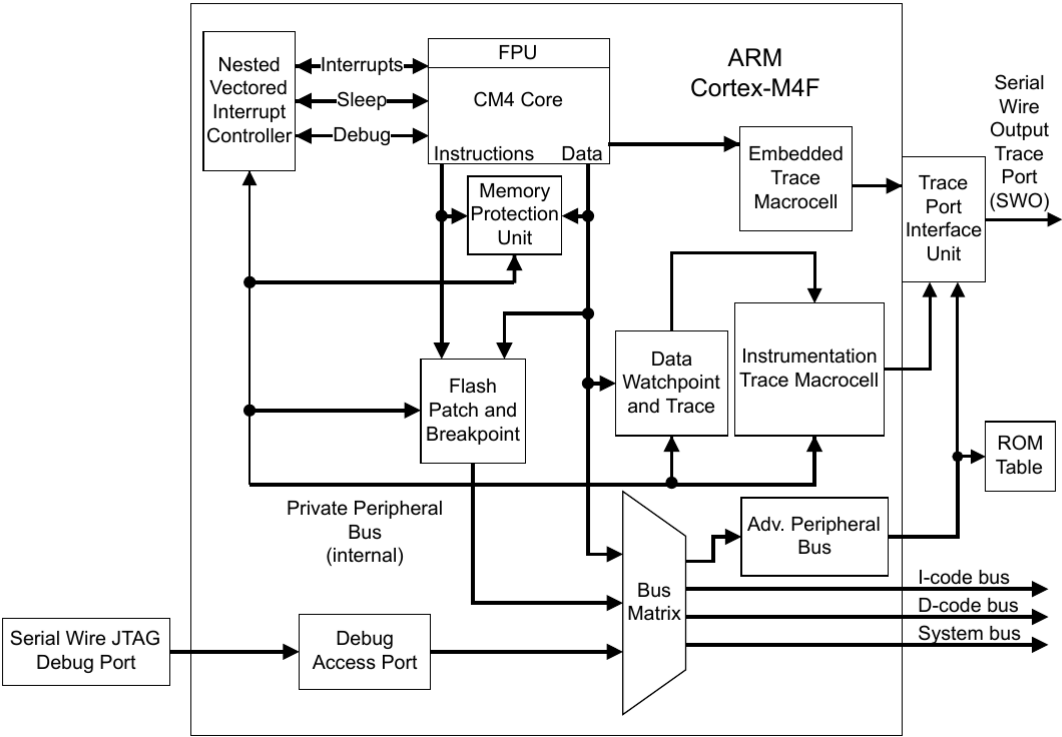
- IEEE 1149.1-1990 compatible Test Access Port (TAP) controller
- Four-bit Instruction Register (IR) chain for storing JTAG instructions
- IEEE standard instructions: BYPASS, IDCODE, SAMPLE/PRELOAD, and EXTEST
- ARM additional instructions: APACC, DPACC and ABORT
- Integrated ARM Serial Wire Debug (SWD)
 - Serial Wire JTAG Debug Port (SWJ-DP)
 - Flash Patch and Breakpoint (FPB) unit for implementing breakpoints
 - Data Watchpoint and Trace (DWT) unit for implementing watchpoints, trigger resources, and system profiling
 - Instrumentation Trace Macrocell (ITM) for support of printf style debugging
 - Embedded Trace Macrocell (ETM) for instruction trace capture
 - Trace Port Interface Unit (TPIU) for bridging to a Trace Port Analyzer

Pin Diagram



CPU Block Diagram

Figure 2-1. CPU Block Diagram



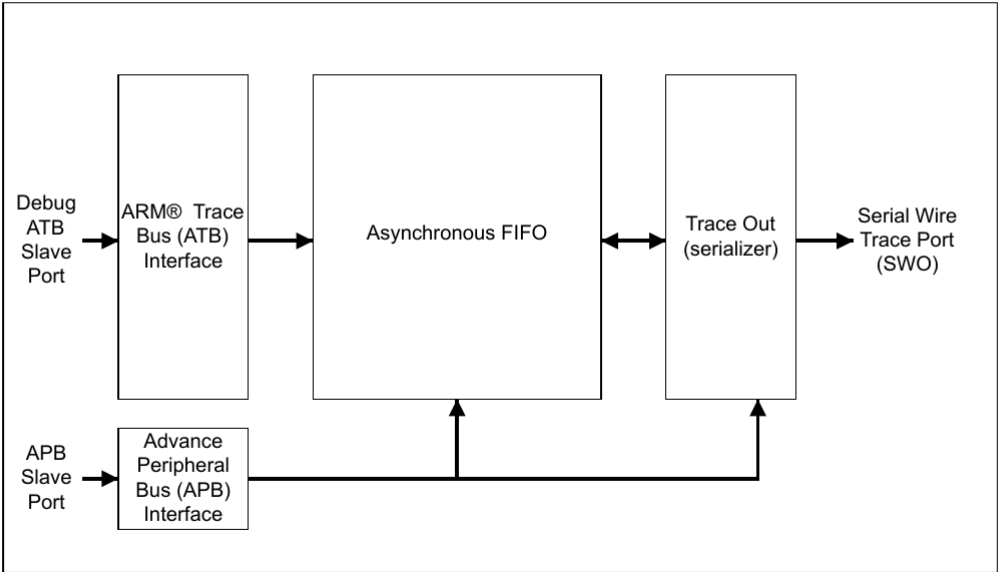
ETM

See ARM® Embedded Trace Macrocell Architecture Specification for more details

Trace Port Interface Unit (TPIU)

The TPIU acts as a bridge between the Cortex-M4F trace data from the ITM, and an off-chip Trace Port Analyzer, as shown in Figure 2-2 on page 72.

Figure 2-2. TPIU Block Diagram



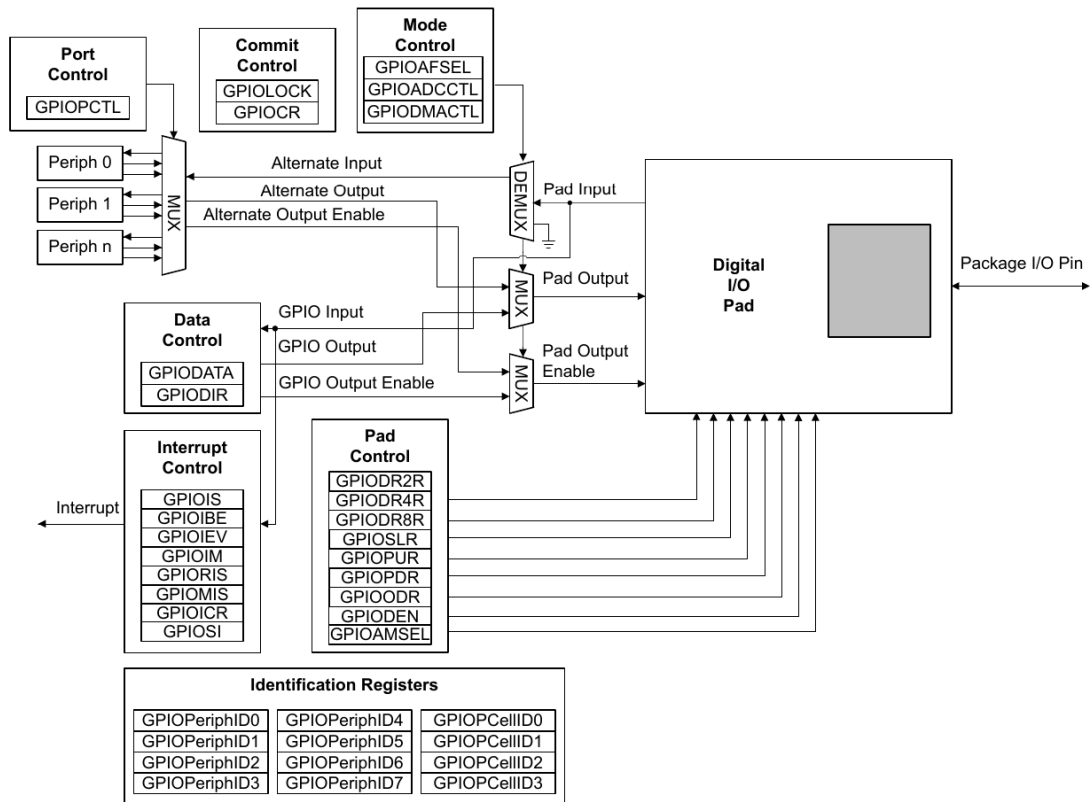
Pin Diagram (See pg. 650)

IO	Pin	Analog Function	1	2	3	4	5	6	7	8	9	14	15
PA0	17	-	U0Rx	-	-	-	-	-	-	CAN1Rx	-	-	-
PA1	18	-	U0Tx	-	-	-	-	-	-	CAN1Tx	-	-	-

IO	Pin	Analog Function	1	2	3	4	5	6	7	8	9	14	15
PA2	19	-	-	SSI0Clk	-	-	-	-	-	-	-	-	-
PA3	20	-	-	SSI0Fss	-	-	-	-	-	-	-	-	-
PA4	21	-	-	SSI0Rx	-	-	-	-	-	-	-	-	-
PA5	22	-	-	SSI0Tx	-	-	-	-	-	-	-	-	-
PA6	23	-	-	-	I2C1SCL	-	M1PWM2	-	-	-	-	-	-
PA7	24	-	-	-	I2C1SDA	-	M1PWM3	-	-	-	-	-	-
PB0	45	USB0ID	U1Rx	-	-	-	-	-	T2CCP0	-	-	-	-
PB1	46	USB0VBUS	U1Tx	-	-	-	-	-	T2CCP1	-	-	-	-
PB2	47	-	-	-	I2C0SCL	-	-	-	T3CCP0	-	-	-	-
PB3	48	-	-	-	I2C0SDA	-	-	-	T3CCP1	-	-	-	-
PB4	58	AIN10	-	SSI2Clk	-	M0PWM2	-	-	T1CCP0	CAN0Rx	-	-	-
PB5	57	AIN11	-	SSI2Fss	-	M0PWM3	-	-	T1CCP1	CAN0Tx	-	-	-
PB6	1	-	-	SSI2Rx	-	M0PWM0	-	-	T0CCP0	-	-	-	-
PB7	4	-	-	SSI2Tx	-	M0PWM1	-	-	T0CCP1	-	-	-	-
PC0	52	-	TCK SWCLK	-	-	-	-	-	T4CCP0	-	-	-	-
PC1	51	-	TMS SWDIO	-	-	-	-	-	T4CCP1	-	-	-	-
PC2	50	-	TDI	-	-	-	-	-	T5CCP0	-	-	-	-
PC3	49	-	TDO SWO	-	-	-	-	-	T5CCP1	-	-	-	-
PC4	16	C1-	U4Rx	U1Rx	-	M0PWM6	-	IDX1	WT0CCP0	U1RTS	-	-	-
PC5	15	C1+	U4Tx	U1Tx	-	M0PWM7	-	PhA1	WT0CCP1	U1CTS	-	-	-
PC6	14	C0+	U3Rx	-	-	-	-	PhB1	WT1CCP0	USB0EPEN	-	-	-
PC7	13	C0-	U3Tx	-	-	-	-	-	WT1CCP1	USB0PFLT	-	-	-
PD0	61	AIN7	SSI3Clk	SSI1Clk	I2C3SCL	M0PWM6	M1PWM0	-	WT2CCP0	-	-	-	-
PD1	62	AIN6	SSI3Fss	SSI1Fss	I2C3SDA	M0PWM7	M1PWM1	-	WT2CCP1	-	-	-	-
PD2	63	AIN5	SSI3Rx	SSI1Rx	-	M0FAULT0	-	-	WT3CCP0	USB0EPEN	-	-	-
PD3	64	AIN4	SSI3Tx	SSI1Tx	-	-	-	IDX0	WT3CCP1	USB0PFLT	-	-	-
PD4	43	USB0DM	U6Rx	-	-	-	-	-	WT4CCP0	-	-	-	-
PD5	44	USB0DP	U6Tx	-	-	-	-	-	WT4CCP1	-	-	-	-
PD6	53	-	U2Rx	-	-	M0FAULT0	-	PhA0	WT5CCP0	-	-	-	-
PD7	10	-	U2Tx	-	-	-	-	PhB0	WT5CCP1	NMI	-	-	-
PE0	9	AIN3	U7Rx	-	-	-	-	-	-	-	-	-	-
PE1	8	AIN2	U7Tx	-	-	-	-	-	-	-	-	-	-
PE2	7	AIN1	-	-	-	-	-	-	-	-	-	-	-
PE3	6	AIN0	-	-	-	-	-	-	-	-	-	-	-
PE4	59	AIN9	U5Rx	-	I2C2SCL	M0PWM4	M1PWM2	-	-	CAN0Rx	-	-	-
PE5	60	AIN8	U5Tx	-	I2C2SDA	M0PWM5	M1PWM3	-	-	CAN0Tx	-	-	-
PF0	28	-	U1RTS	SSI1Rx	CAN0Rx	-	M1PWM4	PhA0	T0CCP0	NMI	C0o	-	-
PF1	29	-	U1CTS	SSI1Tx	-	-	M1PWM5	PhB0	T0CCP1	-	C1o	TRD1	-
PF2	30	-	-	SSI1Clk	-	M0FAULT0	M1PWM6	-	T1CCP0	-	-	TRD0	-
PF3	31	-	-	SSI1Fss	CAN0Tx	-	M1PWM7	-	T1CCP1	-	-	TRCLK	-
PF4	5	-	-	-	-	-	M1FAULT0	IDX0	T2CCP0	USB0EPEN	-	-	-

a. The digital signals that are shaded gray are the power-on default values for the corresponding GPIO pin. Encodings 10-13 are not used on this device.

Digital I/O Pads



Analog/Digital I/O Pads

